PROJECT DESIGN DOCUMENT

PREPAID IoT SIM POOLED DATA PACK ENGINE

ECE-005 | Team E025

A Python-based command-line system for managing fleet SIM accounts, shared prepaid data pools, fair-use limits, pack validity, SIM lifecycle events and usage billing.

Project type	Command-line application / data-processing engine
Primary language	Python
Data format	CSV input and output files
Prepared by	Team E025
Document date	11 November 2026

1. Project Overview

The Prepaid IoT SIM Pooled Data Pack Engine is a Python command-line application designed to manage prepaid mobile-data usage for multiple fleet accounts. Each account can contain several IoT SIM cards. Instead of assigning a separate data balance to every SIM, the system maintains a shared data pool formed from the account's valid purchased packs.

The application reads account, SIM, pack and usage information from CSV files. It executes dated commands such as pack purchases, SIM activation, suspension, resumption and re-registration. It then processes daily usage, applies fair-use and validity rules, calculates overage charges and exports account statements and audit reports.

2. Problem Statement

Managing data consumption across a fleet of IoT SIMs can become difficult when multiple SIMs share prepaid data packs. A system must ensure that data is deducted in the correct order, usage is accepted only for eligible SIMs and valid packs, excessive usage is billed consistently, and invalid activities are recorded for audit. Manual tracking can lead to incorrect balances, unfair allocation and billing errors.

3. Objectives

• Maintain multiple customer accounts and the SIM cards registered under each account.

• Support prepaid pack creation/purchase, validity dates and oldest-valid-pack-first consumption.

• Provide a shared account-level data pool with a per-SIM fair-use limit.

• Manage SIM registration, activation, suspension, resumption, expiry and deactivation.

• Calculate overage charges, GST and final account statements.

• Record rejected commands and usage rows, and export traceable CSV reports.

4. Scope and Assumptions

• The current implementation is a command-line application; it is not a mobile app or a live connection to physical charging/network hardware.

• Input and output data are stored in CSV files. The command file controls the order of dated operations.

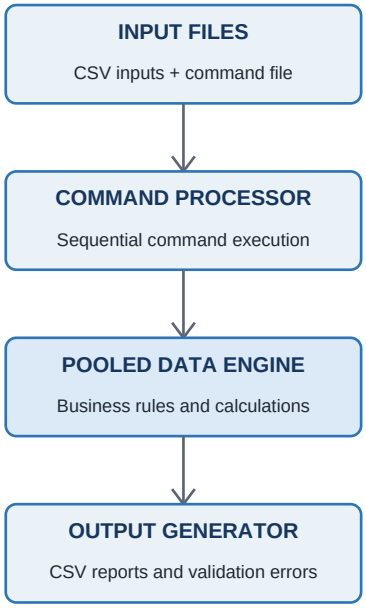
• The supplied sample dataset contains three fleet accounts, 40 SIMs, two pack types and daily usage records spanning 1 October to 14 November 2026.

• A pack's purchase date counts as validity day 1. A 30-day pack purchased on 1 October remains valid through 30 October.

• The rules in the project brief and README are treated as the source of business-rule requirements.

5. System Architecture

The system follows a simple file-driven processing architecture. Input CSV files and a command file are read by the application. The command processor applies dated operations, while the pooled data engine performs validation, state management, data allocation and billing. Results are written to separate CSV output files.



High-level logical architecture of the command-line application

Processing sequence

• Load account, SIM and pack catalogue records.

• Load and validate daily usage rows; identify duplicates and malformed records.

• Read commands in file order and apply the commands for each date before that date's usage is processed.

• For each date, update SIM states, process eligible usage, deduct shared pool data and record overage.

• Generate daily pool logs, final SIM states, error records and account statements.

6. Main Functional Modules

Module	Responsibility
Account management	Loads and stores account details; associates SIMs and packs with the correct account.
SIM management	Registers SIMs and manages ACTIVE, SUSPENDED, EXPIRED, DEACTIVATED and registered/inactive states.
Pack management	Loads pack catalogue data, validates purchases and calculates purchase and expiry dates.
Usage processing	Validates usage rows, rejects invalid SIM activity and processes daily usage in date order.
Pool and fair-use engine	Maintains the shared account pool, consumes oldest valid packs first and enforces the 20% per-SIM fair-use allowance.

Module	Responsibility
Billing and statements	Rounds daily overage per SIM, applies pack rates, adds 18% GST and prepares statements.
Reporting and error handling	Exports pool logs, SIM states, statements and rejected-operation details to CSV.

7. Core Business Rules

Rule	System behaviour
Shared data pool	All valid purchased packs for an account contribute to the account's available pool on a given date.
Pack consumption	The oldest currently valid pack is consumed first. Unused data lapses when that pack expires.
Pack validity	Purchase date is day 1; a 30-day pack purchased on 1 October is usable through 30 October.
Fair use	Each SIM can use up to 20% of the account's purchased pool in the applicable cycle from the oldest currently valid pack's purchase date.
Overage	Usage beyond the SIM's remaining fair-use allowance or available shared pool is treated as overage.
Overage rounding	Overage is accumulated per SIM per day and rounded upward to a whole MB before applying the rate. Example: 35.2 MB is billed as 36 MB.
SIM suspension	Suspended SIMs do not consume data; usage rows for them are rejected.
Grace period	A pack bought within seven days after expiry can reactivate eligible expired SIMs. After the grace period, a deactivated SIM must be registered again and activated.
Billing	Pack cost and overage charge form the subtotal. GST is 18%; the final statement amount is rounded to the nearest rupee as specified.
Error recording	Invalid commands and rejected usage rows are recorded with source, line number and reason.

8. Input and Output Files

File	Purpose
accounts.csv	Account identifiers and customer details.
sims.csv	SIM ICCIDs, account association, device type and registration date.
pack_types.csv	Pack size, validity, price and overage rate.
usage_original.csv	Daily SIM data usage records; expected columns: date, iccid, mb_used.
commands.txt	Ordered operations such as loading files, buying packs, changing SIM states and requesting statements.
pool_log.csv	Daily account-level pool balance, accepted usage, pool deduction, overage and active SIM count.
statements.csv	Account billing summary including pack cost, overage, GST and total.
sim_states.csv	Final state and state-change date for each SIM.

File	Purpose
errors.csv	Rejected commands and usage rows with reason codes.

9. Technologies and Implementation

• **Python:** implements the command-line workflow, business rules, date handling, calculations and CSV processing.

• **CSV:** provides lightweight file-based input, storage and reporting for the supplied dataset.

• **Python standard libraries:** csv for reading/writing records, datetime/timedelta for validity and grace-period dates, and math.ceil for per-SIM daily overage rounding.

• **VS Code:** used as the development environment.

• **Git and GitHub:** used for source-code version control and repository sharing.

10. Validation and Testing

The project includes unit tests for account/SIM registration, pack creation and purchase, shared-pool deduction, fair-use overage, SIM suspension/resumption and statement/GST calculations. The latest run reported 11 tests passing. The command-driven sample application was also run through the supplied period and reported successful completion.

Important sample scenarios include:

• Usage on the last valid day is accepted; usage after pack expiry is rejected.

• When two packs overlap, the older valid pack is consumed first.

• A SIM reaching its fair-use limit is billed for further usage even when the account pool still has data.

• Suspended, inactive, unknown and duplicate usage records are rejected and logged.

• A pack purchase on grace-period day 7 can reactivate eligible SIMs; a purchase on day 8 does not reverse deactivation.

11. Expected Benefits

• Improves accuracy and consistency of pooled data allocation.

• Reduces manual effort in tracking pack validity, SIM states and usage.

• Provides transparent overage and GST calculations.

• Creates an audit trail through daily logs and error reports.

• Can serve as a foundation for a future database-backed or user-interface-based fleet data management application.

12. Limitations and Future Enhancements

• The current system uses CSV files and a command-line interface; it does not provide a graphical dashboard or multi-user access control.

• The current sample processing is based on supplied records and commands, not live network telemetry from IoT SIM providers.

• Future work may include a database, web/mobile dashboard, authentication, API integration with telecom providers, live usage updates, alerts and downloadable invoices.

13. Conclusion

The Prepaid IoT SIM Pooled Data Pack Engine demonstrates how a software-based system can manage multiple fleet accounts and SIMs using shared prepaid data packs. By combining pack validity, oldest-first pool deduction, fair-use enforcement, SIM lifecycle management, overage billing and structured error reporting, the application provides a consistent and auditable approach to pooled IoT data management.

Document note: This design document describes the command-line implementation and the business rules recorded in the project's README and decision log. Before final submission, confirm that the local README, commands.txt and input filenames match the files used in the final demonstration.